Exp: 1

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Aim

To implement and demonstrate the Find-S algorithm to find the most specific hypothesis based on a given set of training data samples.

Algorithm

Start.

Load the training dataset.

Initialize the hypothesis with the most specific values.

Read each training example.

If the example is positive:

If the hypothesis is empty, initialize it using the example.

Otherwise, compare each attribute with the hypothesis.

If an attribute differs, replace its value with ?.

Ignore negative examples.

Display the final hypothesis.

Stop.

Program

```
training_data = [
    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
    ['Rainy', 'Mild', 'High', 'Weak', 'Yes'],
    ['Rainy', 'Cool', 'Normal', 'Weak', 'Yes'],
```

```
    ['Rainy', 'Cool', 'Normal', 'Strong', 'No'],
    ['Overcast', 'Mild', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Mild', 'High', 'Weak', 'No']
]

# Initialize hypothesis
hypothesis = None

print("Find-S Algorithm")
print("----------------")

# Process training examples
for example in training_data:

    # Consider only positive examples
    if example[-1] == 'Yes':

        attributes = example[:-1]

        if hypothesis is None:
            hypothesis = attributes.copy()
        else:
            for i in range(len(hypothesis)):
                if hypothesis[i] != attributes[i]:
                    hypothesis[i] = '?'

        print("Positive Example:", attributes)
        print("Updated Hypothesis:", hypothesis)
```

```
# Display final hypothesis
print("\nFinal Most Specific Hypothesis:")
print(hypothesis)
```

OUTPUT

Find-S Algorithm

Positive Example: ['Overcast', 'Hot', 'High', 'Weak']

Updated Hypothesis: ['Overcast', 'Hot', 'High', 'Weak']

Positive Example: ['Rainy', 'Mild', 'High', 'Weak']

Updated Hypothesis: ['?', '?', 'High', 'Weak']

Positive Example: ['Rainy', 'Cool', 'Normal', 'Weak']

Updated Hypothesis: ['?', '?', '?', 'Weak']

Positive Example: ['Overcast', 'Mild', 'Normal', 'Strong']

Updated Hypothesis: ['?', '?', '?', '?']

Final Most Specific Hypothesis:

['?', '?', '?', '?']

Exp : 2

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples

Aim

To implement and demonstrate the Candidate Elimination algorithm in Python using training examples stored in a CSV file and find the set of hypotheses consistent with the training examples.

Algorithm

Start.

Load the training dataset from a CSV file.

Initialize S to the most specific hypothesis.

Initialize G to the most general hypothesis.

For every positive example:

Remove hypotheses from G that do not cover the example.

Generalize S to cover the example.

For every negative example:

Remove hypotheses from S that cover the example.

Specialize G to exclude the example.

Remove inconsistent and redundant hypotheses.

Display the final S and G boundaries.

Stop.

progaram

```
import csv

# Create CSV file
data = [
    ["Weather", "Temperature", "Humidity", "Wind", "EnjoySport"],
    ["Sunny", "Warm", "Normal", "Weak", "Yes"],
    ["Sunny", "Warm", "High", "Strong", "Yes"],
```

```
    ["Rainy", "Cold", "High", "Weak", "No"],
    ["Sunny", "Cold", "Normal", "Weak", "Yes"],
    ["Rainy", "Warm", "Normal", "Strong", "No"]
]

with open("training_data.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(data)

print("CSV file created successfully!")

# Read CSV file
with open("training_data.csv", "r") as f:
    reader = csv.reader(f)
    rows = list(reader)

print("\nTraining Data:")
for row in rows:
    print(row)
```

Output

```
CSV file created successfully!
Training Data:
['Weather', 'Temperature', 'Humidity', 'Wind', 'EnjoySport']
['Sunny', 'Warm', 'Normal', 'Weak', 'Yes']
['Sunny', 'Warm', 'High', 'Strong', 'Yes']
['Rainy', 'Cold', 'High', 'Weak', 'No']
['Sunny', 'Cold', 'Normal', 'Weak', 'Yes']
['Rainy', 'Warm', 'Normal', 'Strong', 'No']
```

EXP: 3

Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample

Aim

To demonstrate the working of the ID3 algorithm by constructing a decision tree using a suitable dataset and classifying a new sample.

Algorithm

1. Start.
2. Load the training dataset.
3. Calculate the entropy of the target class.
4. Calculate the information gain for each attribute.
5. Select the attribute with the highest information gain.
6. Create a decision node using that attribute.
7. Repeat the process until the classes are classified.
8. Use the decision tree to predict a new sample.
9. Stop

PROGARAM

```
# Experiment 3: Decision Tree using ID3

from sklearn.tree import DecisionTreeClassifier, export_text

# Training data

# Outlook: Sunny=0, Overcast=1, Rainy=2

# Temperature: Hot=0, Mild=1, Cool=2

# Humidity: High=0, Normal=1

# Wind: Weak=0, Strong=1
```

```
X = [
    [0, 0, 0, 0],
    [0, 0, 0, 1],
    [1, 0, 0, 0],
    [2, 1, 0, 0],
    [2, 2, 1, 0],
    [2, 2, 1, 1],
    [1, 1, 1, 1],
    [0, 1, 0, 0]
]

# 0 = No, 1 = Yes
y = [0, 0, 1, 1, 1, 0, 1, 0]

# Create the decision tree
model = DecisionTreeClassifier(
    criterion="entropy",
    random_state=0
)

# Train the model
model.fit(X, y)

# Display decision tree
features = ["Outlook", "Temperature", "Humidity", "Wind"]

print("Decision Tree:")
print(export_text(model, feature_names=features))
```

```
# Classify a new sample
sample = [[1, 1, 1, 0]]

prediction = model.predict(sample)

if prediction[0] == 1:
    print("Prediction: Play Tennis = Yes")
else:
    print("Prediction: Play Tennis = No")
```

Output

```
Decision Tree:
|--- Outlook <= 0.50
|   |--- class: 0
|--- Outlook >  0.50
|   |--- Temperature <= 1.50
|   |   |--- class: 1
|   |--- Temperature >  1.50
|   |   |--- Wind <= 0.50
|   |   |   |--- class: 1
|   |   |--- Wind >  0.50
|   |   |   |--- class: 0

Prediction: Play Tennis = Yes
```